Desenvolvimento de Biblioteca Modular Avançada de Tópicos Matemáticos em C

Estrutura, Compilação (Make), Bibliotecas Estática/Dinâmica, Testes e Git

André De Vlieger Lima Lucas Correa Meira Vitor Wallace de Moraes Alves

Instituto de Matemática e Estatística e Ciência da Computação

19 de Outubro de 2026

1. Descrição do Problema e Domínio

Objetivo do Projeto

Criar um ecossistema coeso, modular e reutilizável de ferramentas matemáticas e geométricas na linguagem C, voltado para cálculo computacional e ambientes gráficos

Domínio Abrangido (5 Módulos):

- **vetores**: Operações vetoriais em 2D, 3D e ND;
- **geometria**: Cadarço (área), Ray-Casting e volumes 3D;
- **calculo**: Diferenças finitas, gradientes e integração numérica;
- **calculo_numerico**: Raízes (Bissecção e Newton-Raphson);
- **polinomios**: Polinômios em listas encadeadas (qualquer grau).

2. Arquitetura do Software

Princípios de Design:

- Encapsulamento estrito e visibilidade mínima;
- Interfaces públicas isoladas na pasta "include/";
- Implementação interna contida na pasta "src/".

Estrutura de Diretórios

```
projeto/
 |- include/      # Contratos da API (.h)
 |- src/          # Implementação interna (.c)
 |- tests/        # Testes automatizados
 |- app/          # Aplicação cliente (main.c)
 |- build/        # Objetos intermediarios (.o)
 |- lib/          # Bibliotecas geradas (.a e .so)
 |- bin/          # Executáveis finais
 |- Makefile     # Automação de build
 `- README.md     # Documentação e instruções
```

3. Organização e Responsabilidade dos Módulos

- **vetores** (`vetores.h / .c`): Operações em R3 e ND, normas, produtos escalar e vetorial;
- **geometria** (`geometria.h / .c`): Algoritmo Shoelace, Ray-Casting em polígonos, malhas 3D;
- **calculo** (`calculo.h / .c`): Diferenças finitas centradas, gradiente, integrais simples e duplas;
- **calculo_numerico** (`calculo_numerico.h / .c`): Métodos da Bissecção e Newton-Raphson;
- **polinomios** (`polinomios.h / .c`): Polinômios esparsos com alocação dinâmica.

4. Aplicação Cliente (app/main.c)

Respeito Estrito à Interface Pública

A aplicação cliente atua como um consumidor independente da biblioteca.

Regras de Encapsulamento Adotadas:

- Inclusão exclusiva dos cabeçalhos (.h) presentes na pasta include/;
- Proibição total de incluir arquivos da pasta src/;
- Interação única e exclusiva por meio das funções da API pública;
- Nenhuma dependência de membros internos ou structs privadas.

5. Processo de Compilação com Makefile

Principais Alvos Padronizados:

- **make all**: Compila os módulos, gera as bibliotecas e o cliente;
- **make clean**: Remove os artefatos de build/, lib/ e bin/;
- **make run**: Compila (se necessário) e executa a aplicação;
- **make test**: Compila e executa a suíte de testes;
- **make install**: Instala cabeçalhos e bibliotecas em PREFIX.

Compilação Incremental

Cada objeto build/%.o depende exclusivamente do seu fonte src/%.c e do seu cabeçalho include/%.h, evitando recompilações desnecessárias.

6. Geração e Utilização da Biblioteca Estática

Mecanismo da Biblioteca Estática (libapp.a)

O utilitário 'ar' empacota os arquivos objeto (.o). O linker copia o código diretamente para o binário final durante a compilação.

Comandos de Terminal:

Comandos

```
# 1. Compilação dos objetos
gcc -Wall -Wextra -Iinclude -c src/*.c && mv *.o build/

# 2. Empacotamento com 'ar'
ar rcs lib/libapp.a build/*.o

# 3. Compilação e linkagem da Aplicação Cliente
gcc -Iinclude app/main.c lib/libapp.a -lm -o
bin/client_estatico
```

7. Geração e Utilização da Biblioteca Dinâmica

Mecanismo da Biblioteca Dinâmica (libapp.so)

Exige código independente de posição (-fPIC). A resolução dos símbolos ocorre em tempo de execução pelo carregador do sistema operacional.

Comandos de Terminal:

Comandos

```
# 1. Compilação com flag -fPIC
gcc -Wall -Wextra -fPIC -Iinclude -c src/*.c
# 2. Geração do objeto compartilhado (.so)
gcc -shared build/*.o -o lib/libapp.so
# 3. Linkagem e Execucao
gcc -Iinclude app/main.c -Llib -lapp -lm -o
bin/client_dinamico
LD_LIBRARY_PATH=lib ./bin/client_dinamico
```

10. Exemplo de Teste que Falhou e Correção

Falha Identificada no Gradiente

No cálculo da derivada parcial em src/calculo.c, o teste esperava 2.0 para a função $f(x,y)=x^2+y^2$ no ponto (1,1), mas recebeu -2.0 (sinal invertido).

Causa Raiz e Correção em src/calculo.c:

Incorreto (Ordem Invertida)

```
return (l_inferior - l_superior) / (2.0 * h);
```

Corrigido (Ordem Correta: f(x+h) - f(x-h))

```
return (l_superior - l_inferior) / (2.0 * h);
```

Resultado: Após o ajuste, o comando 'make test' aprovou 100% dos módulos.

11. Estratégia de Versionamento com Git

Estratégia do Histórico:

- Desenvolvimento modular em branches separadas para cada funcionalidade;
- Commits atômicos e com mensagens descritivas sobre o progresso;
- Integração do código na branch principal por meio de merges;
- Criação da tag anotada **v1.0** para delimitar a versão final e estável do projeto.

12. Dificuldades Encontradas e Decisões

- **Resolução de Caminhos do .so**:
  Solução: Configuração do rpath (-Wl,-rpath=./lib) no Makefile para facilidade no 'make run'.
- **Gestão de Memória em Listas e Vetores**:
  Solução: Responsabilidade explícita na API (cada função de criação exige sua função correspondente de liberação).
- **Comparação de Ponto Flutuante**:
  Solução: Padronização do uso de margem de tolerância numérica nos testes.

13. Resultados Obtidos e Conclusão

Resultados Finais:

- 5 módulos matemáticos totalmente desenvolvidos e testados.
- Automação completa no Makefile (bibliotecas estática e dinâmica).
- Aplicação cliente demonstrando o uso correto da API pública.
- Histórico do Git estruturado com branches, merges e tag v1.0.